Раздел 1

1. Вход под пользователем user1 из практики 2:

```
root@eltex-practice2-pg2-v2:~# su - user1
Warning: your password will expire in 6 days.
user1@eltex-practice2-pg2-v2:~$ whoami
user1
```

Рисунок 1 – Вход под пользователем user1 и проверка пользователя

2. Количество процессов, имеющих несколько потоков выполнения:

```
user1@eltex-practice2-pg2-v2:~$ ps -eLf | awk '{print $2}' | sort | uniq -c |
 awk '$1 > 1' | wc -l
10
```

Рисунок 2 – Подсчет количества процессов, имеющих несколько потоков выполнения

3. Запуск top и настройка вывода полей с информацией о процессе:

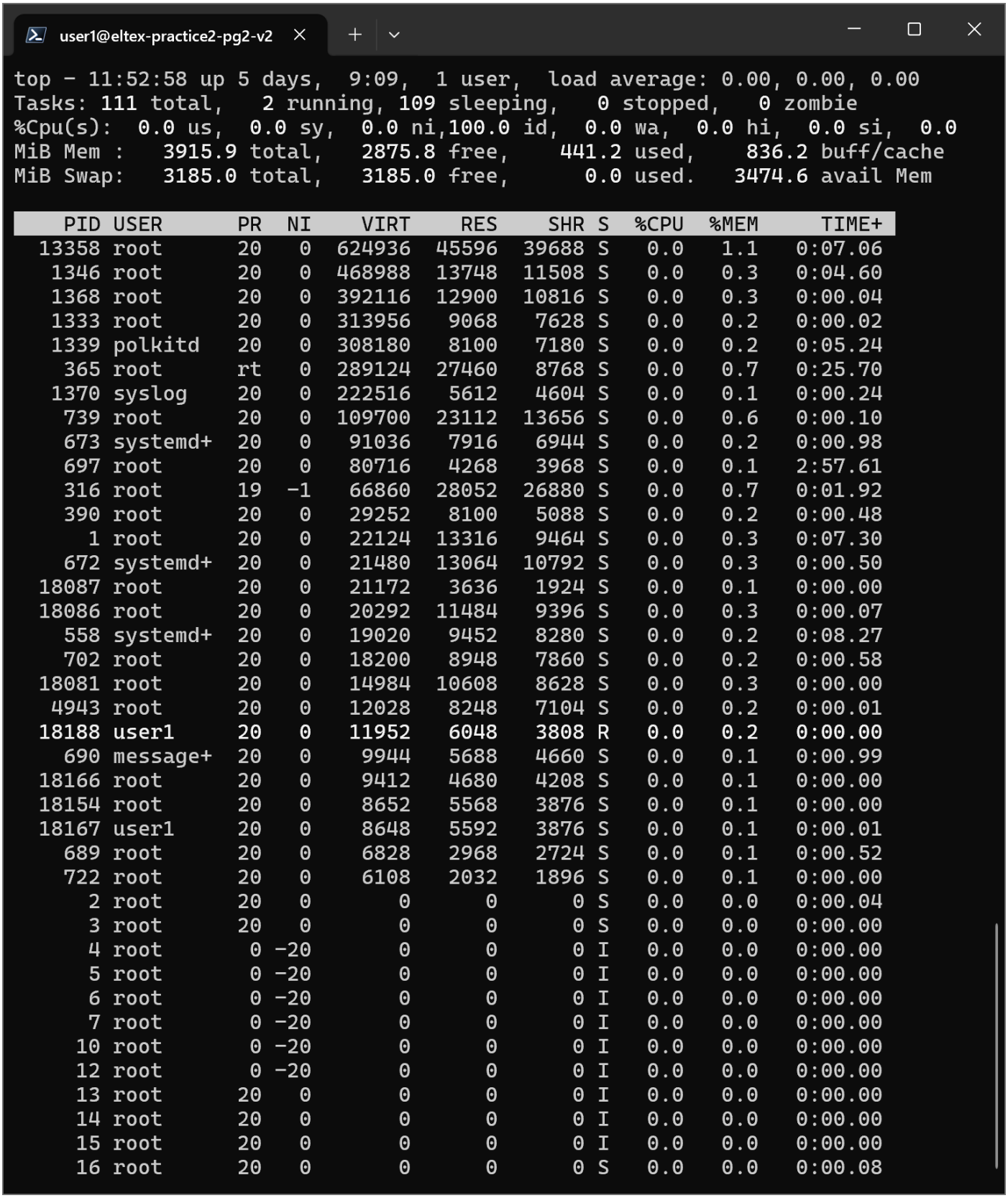


Рисунок 3 – Настройка вывода top: сортировка списка процессов по убыванию значения поля VIRT

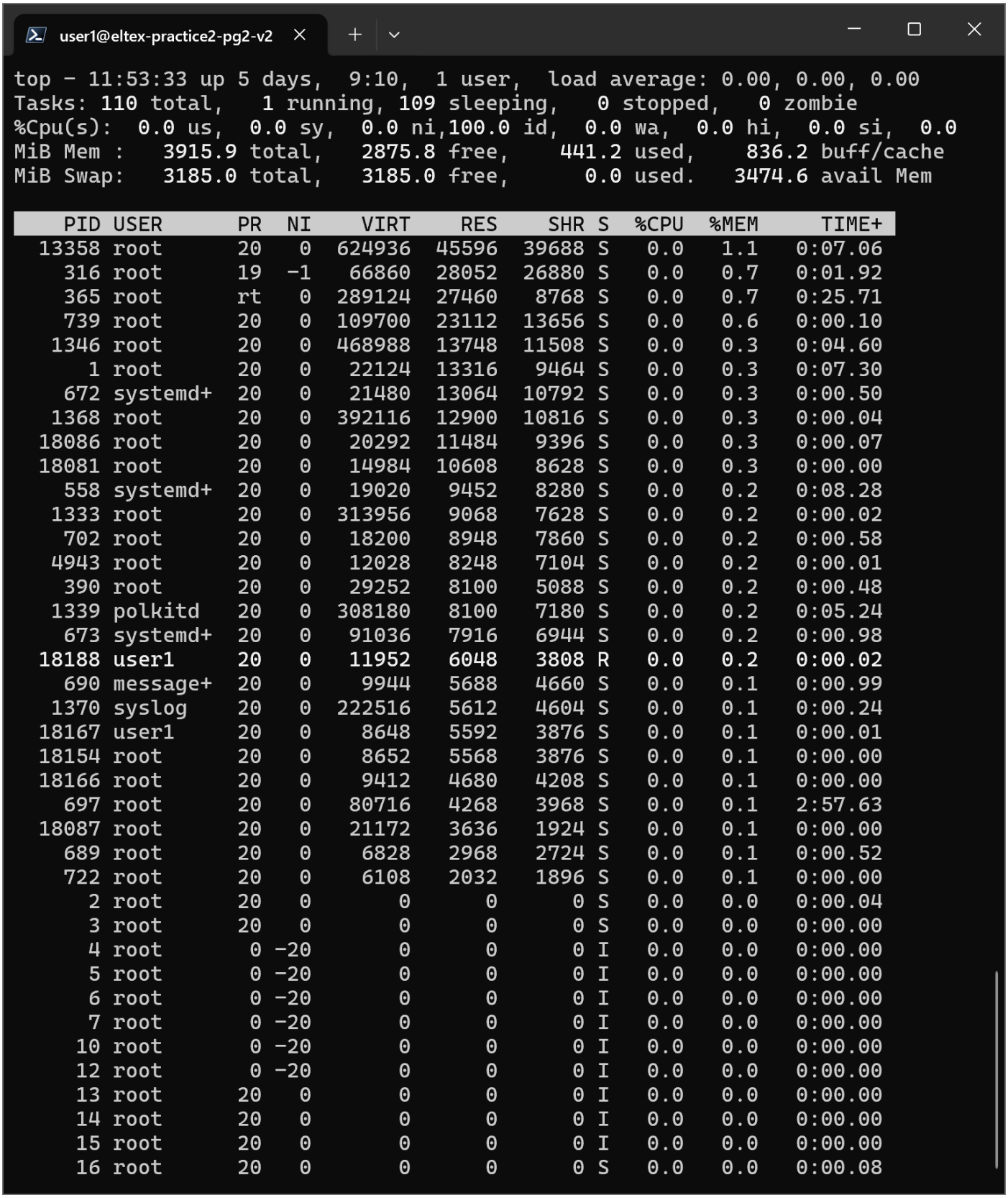


Рисунок 4 – Настройка вывода top: сортировка списка процессов по убыванию значения поля RES

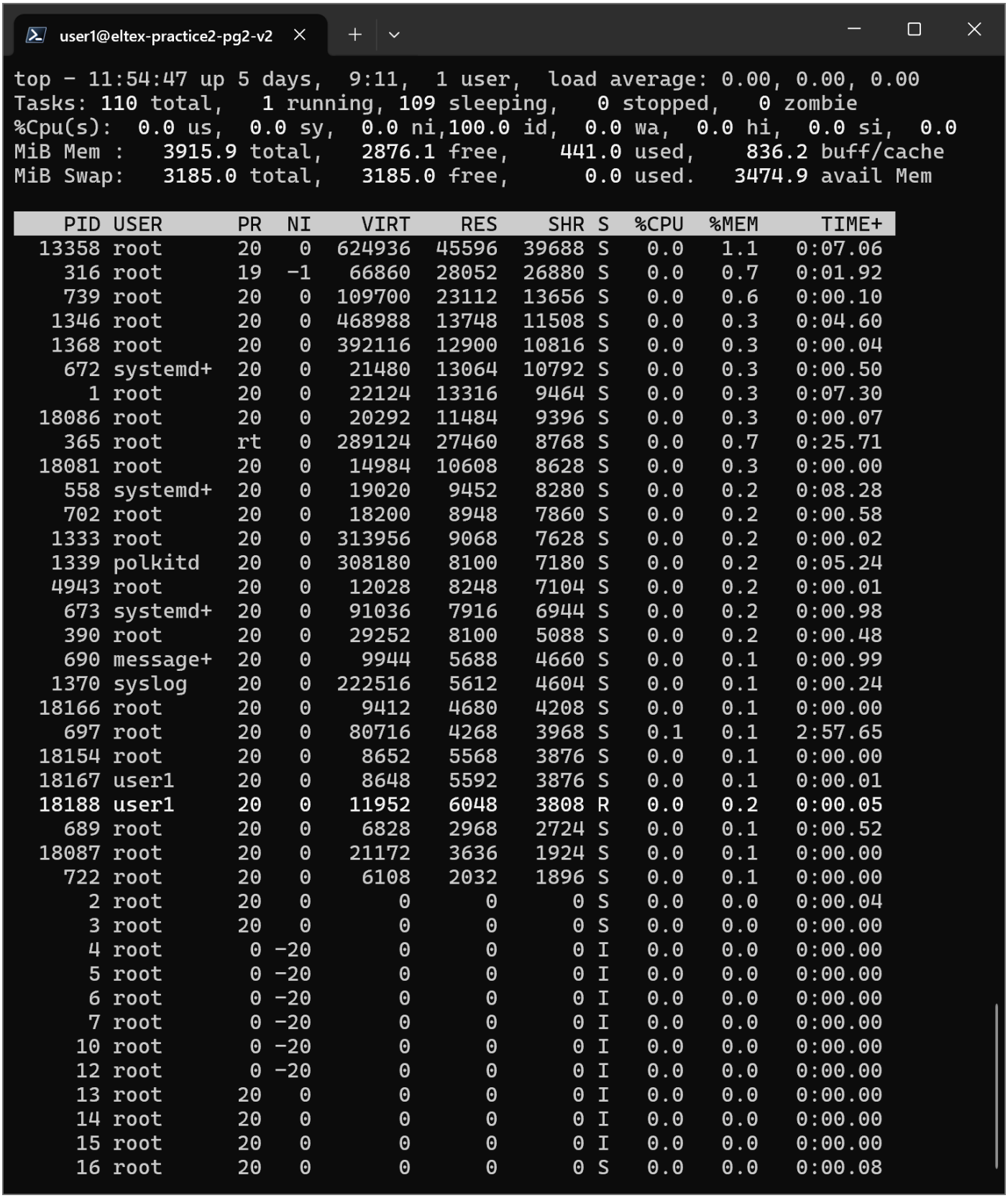


Рисунок 5 – Настройка вывода top: сортировка списка процессов по убыванию значения поля SHR

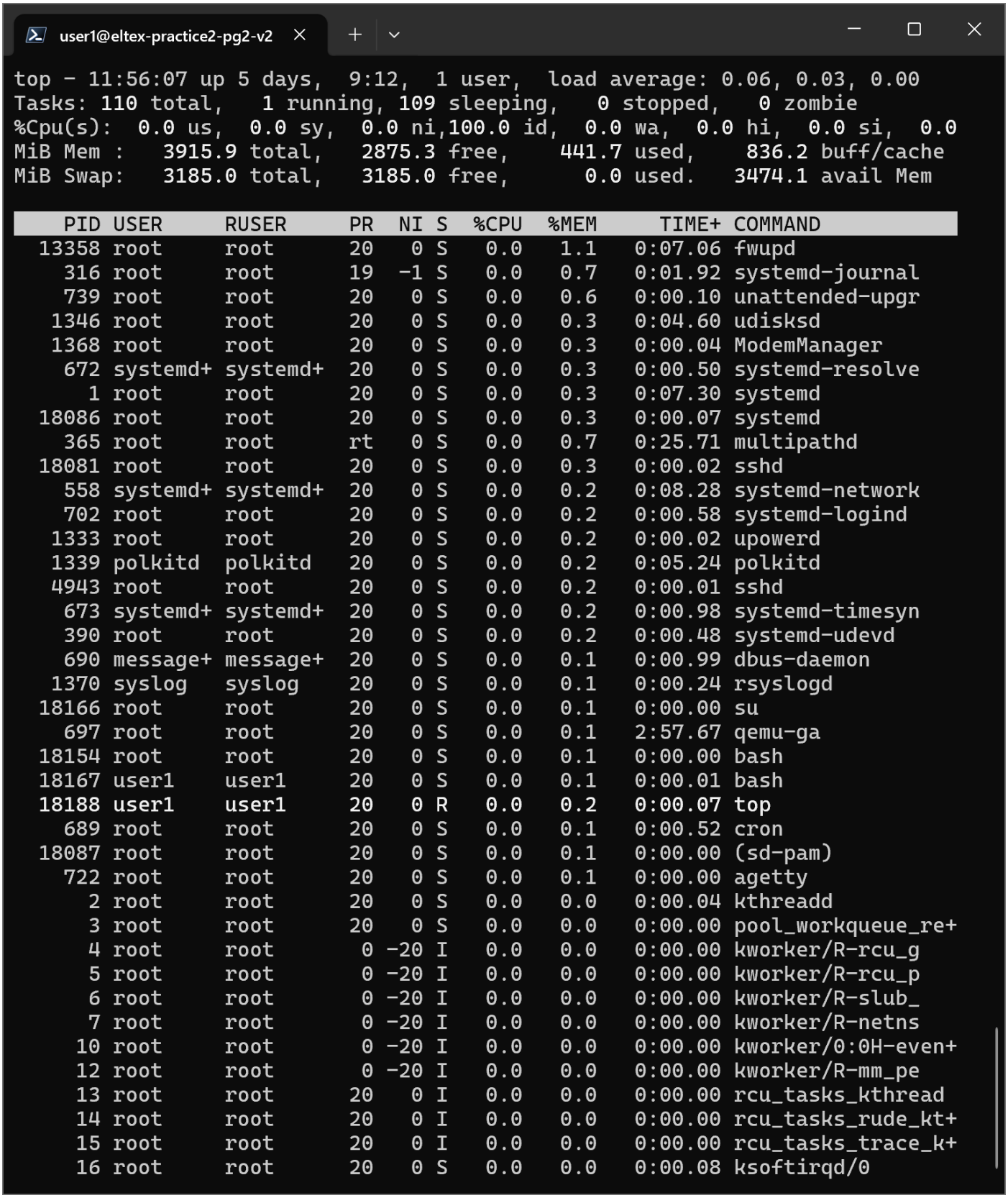


Рисунок 6 – Настройка вывода top: удалены поля VIRT, RES, SHR; добавлено поле RUSER, расположенное после поля USER

4. Выполнение в другом терминальном окне команды passwd:

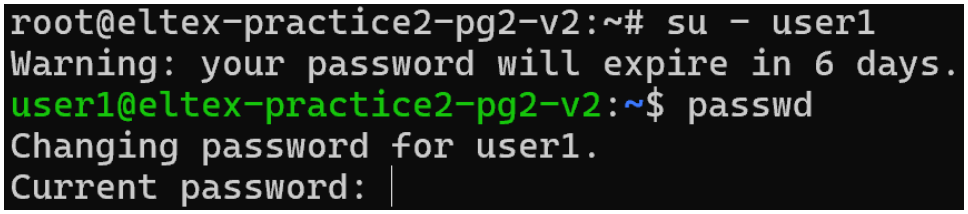


Рисунок 7 – Выполнение команды passwd в другом терминальном окне (состояние запроса текущего пароля)

5. Вывод всех процессов для пользователя user1, процесса, запущенного командой passwd, отправка сигналов этому процессу:

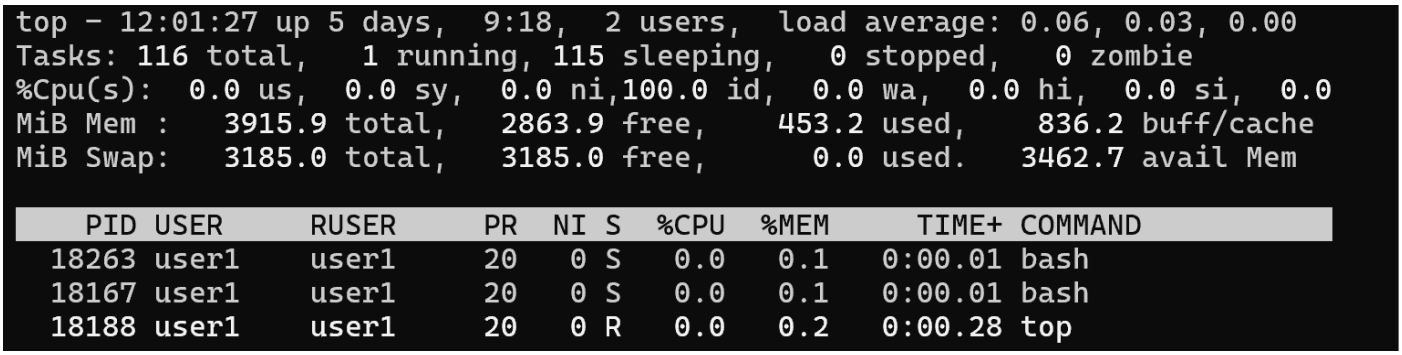


Рисунок 8 – Вывод процессов в top только для реального пользователя user1

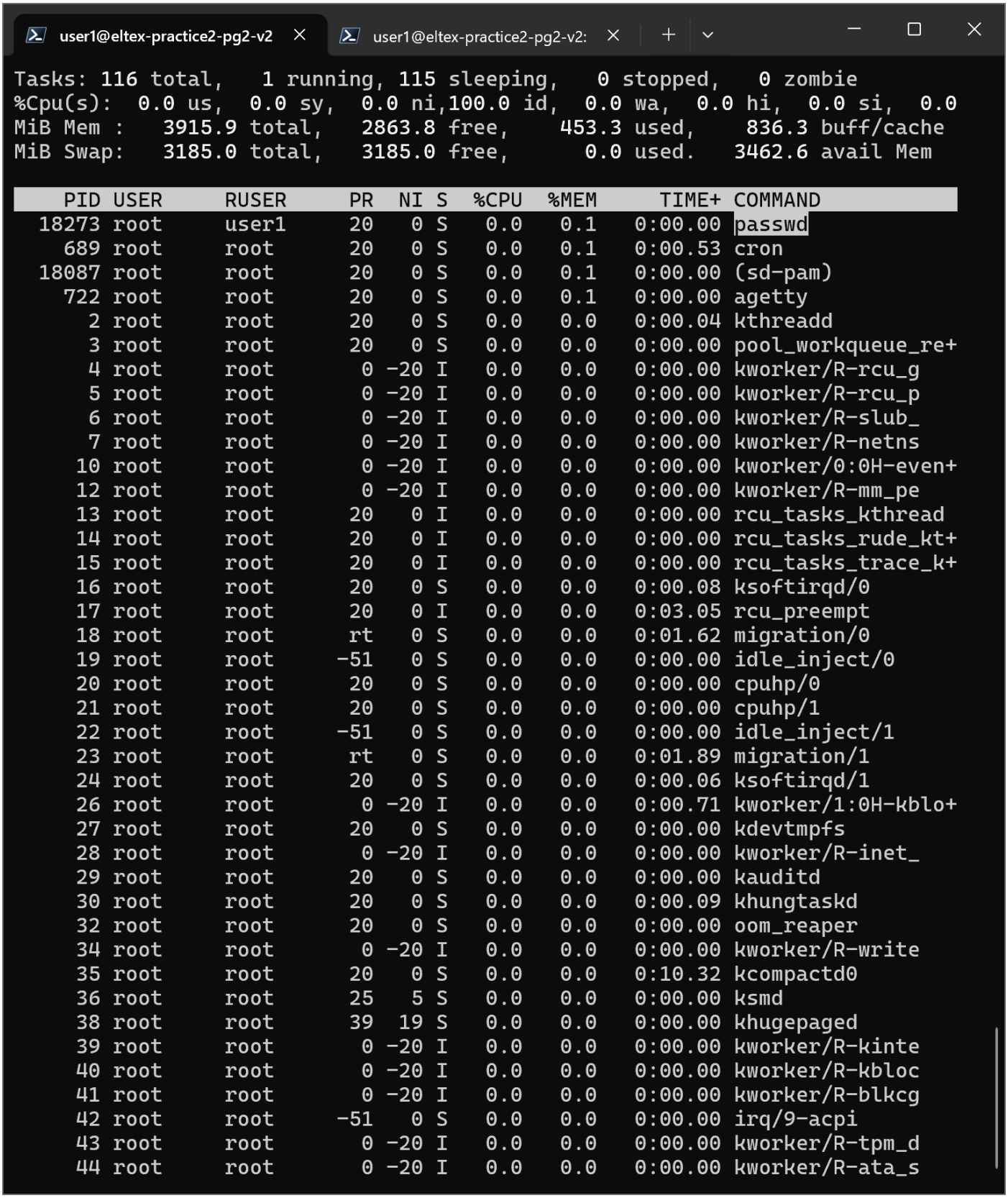


Рисунок 9 – Поиск процесса passwd в top (демонстрация работы SUID: USER – root, RUSER – user1)

```
user1@eltex-practice2-pg2-v2:~$ passwd
Changing password for user1.
Current password: Connection to 217.71.138.1 closed by remote host.
Connection to 217.71.138.1 closed.
```

Рисунок 10 – Отправка сигналов процессу passwd, приведшая к завершению сессии

```
user1@eltex-practice2-pg2-v2:~$ passwd
Changing password for user1.
Current password: Killed
```

Рисунок 11 – Отправка сигналов процессу passwd, приведшая к завершению команды passwd

6. Выполнение команды vim ~/file_task3.txt и нажатие Ctrl + Z:

```
user1@eltex-practice2-pg2-v2:~$ vim ~/file_task3.txt

[1]+  Stopped                 vim ~/file_task3.txt
```

Рисунок 12 – Выполнение команды vim ~/file_task3.txt и нажатие Ctrl + Z

7. Выполнение команды sleep 600, нажатие Ctrl + Z, выполнение команды jobs:

```
user1@eltex-practice2-pg2-v2:~$ sleep 600
^Z
[2]+  Stopped                 sleep 600
user1@eltex-practice2-pg2-v2:~$ jobs
[1]-  Stopped                 vim ~/file_task3.txt
[2]+  Stopped                 sleep 600
```

Рисунок 13 – Выполнение команды sleep 600, нажатие Ctrl + Z, выполнение команды jobs

8. Последнее задание сделать фоновым:

```
user1@eltex-practice2-pg2-v2:~$ sleep 600 &
[1] 18393
```

Рисунок 14 – Запуск команды sleep 600 & для перевода последнего задания в фоновый режим

9. Изменение числа NICE у задания:

```
user1@eltex-practice2-pg2-v2:~$ renice 10 18393
18393 (process ID) old priority 0, new priority 10
```

Рисунок 15 – Изменение числа NICE у задания (sleep 600) с помощью команды renice

10. Проверка изменения числа NICE у задания:

```
user1@eltex-practice2-pg2-v2:~$ ps -l -p 18393
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1001   18393   18352  0  90  10 -  1422 hrtime pts/1    00:00:00 sleep
```

Рисунок 16 – Проверка изменения числа NICE у задания с помощью команды ps -l

11. Сделать задание vim ~/file_task3.txt активным и выйти из редактора:

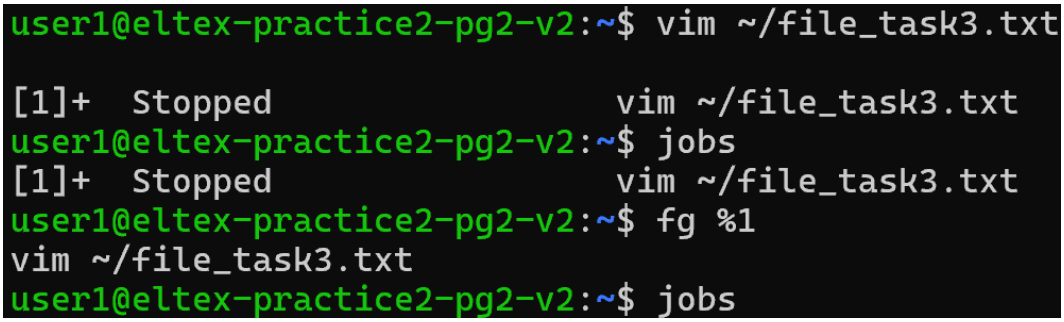


Рисунок 17 – Возврат задания vim в активный режим (fg %1) и выход из редактора

12. Отправка сигнала SIGTERM заданию sleep 600 и выполнение команды jobs:

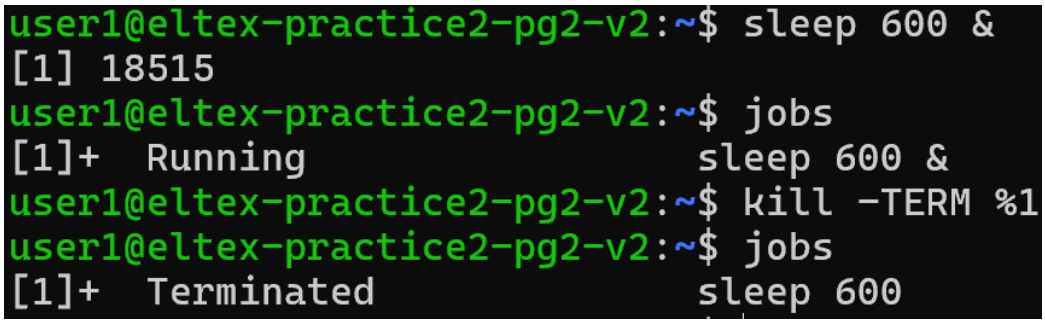


Рисунок 18 – Отправка сигнала SIGTERM заданию sleep 600 (kill -TERM %1) и проверка статуса через jobs

13. Создание перехватчика сигналов SIGINT и SISQUIT внутри командного интерпретатора и отправка сигналов самому себе:

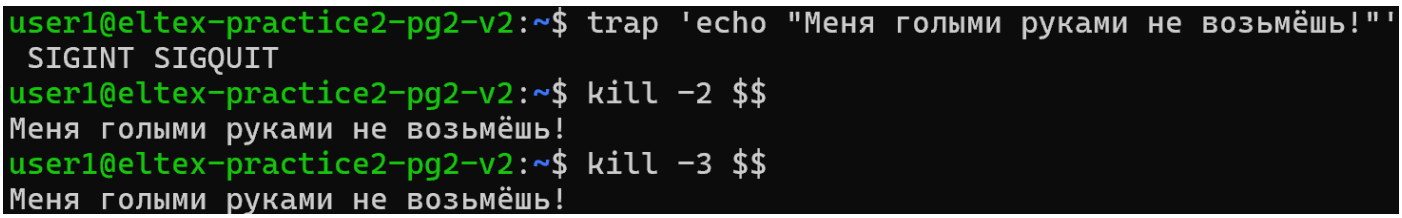


Рисунок 19 – Создание перехватчика сигналов SIGINT и SIGQUIT с помощью trap и отправка сигналов самому себе

14. Создание зомби-процессов:

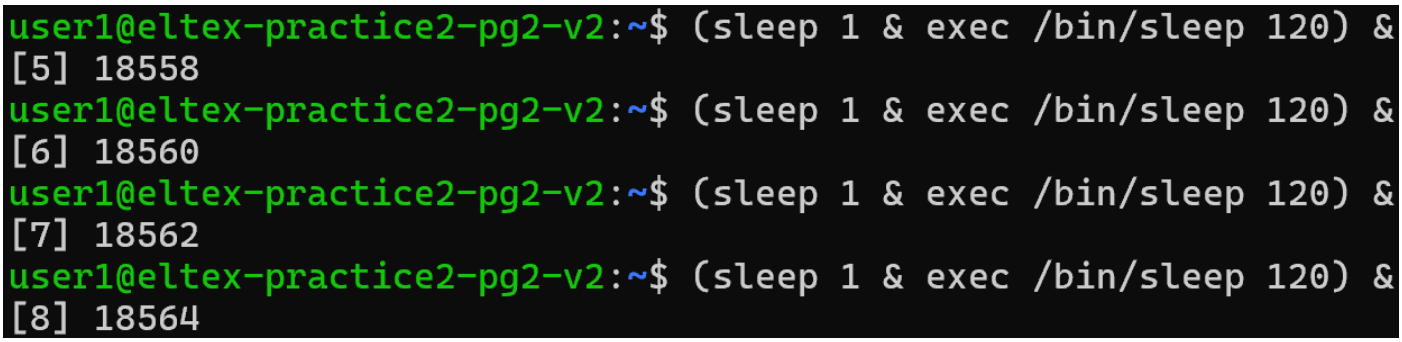


Рисунок 20 – Создание нескольких зомби-процессов с помощью конструкции (sleep 1 & exec /bin/sleep 120) &

15. Поиск с помощью команды ps всех зомби-процессов, вывод их PID и PPID, попытка досрочного завершения выполнения зомби-процессов:

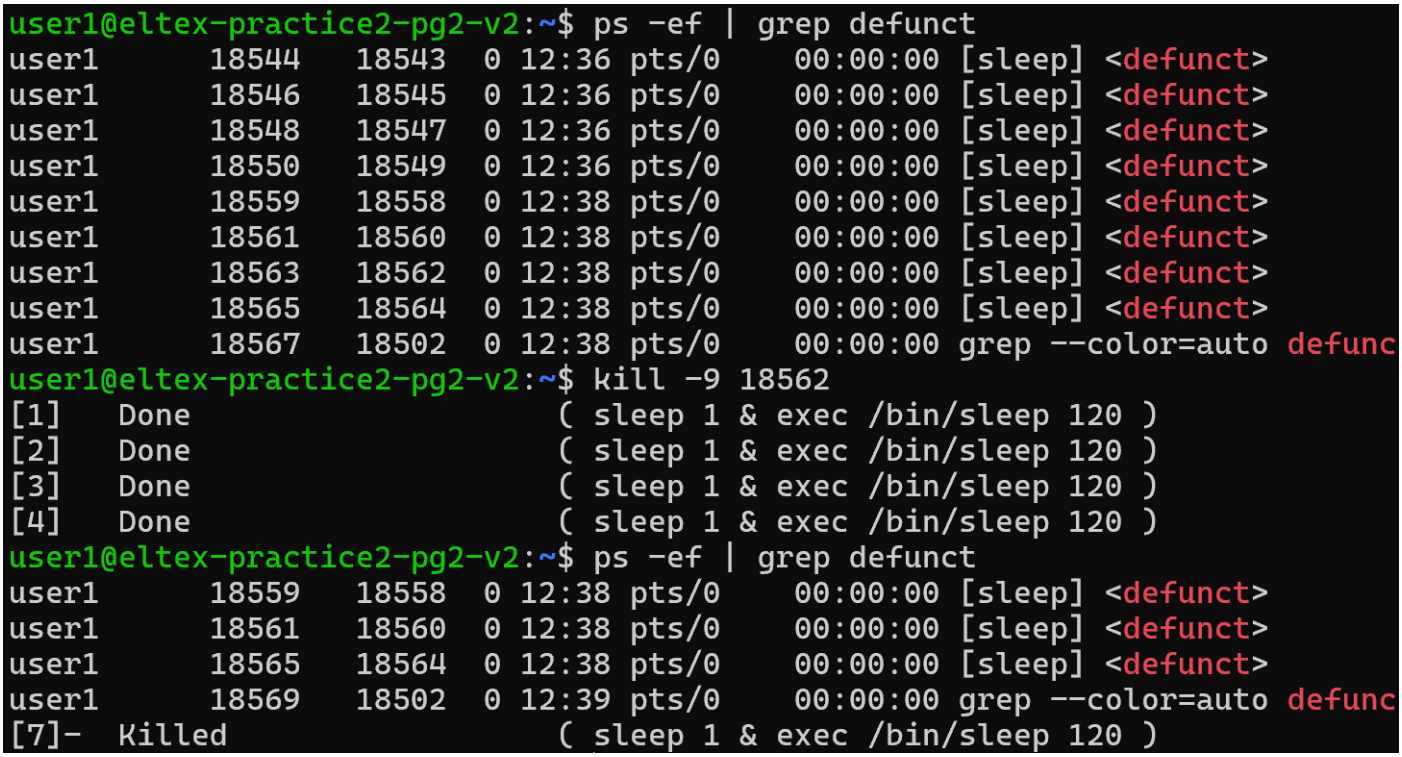


Рисунок 21 – Поиск зомби-процессов (ps -ef | grep defunct), вывод их PID/PPID и попытка досрочного завершения

В ходе выполнения задания были искусственно созданы зомби-процессы с помощью конструкции (sleep 1 & exec /bin/sleep 120) &, где родительский процесс завершается раньше дочернего или не считывает его статус. С помощью команды ps -ef | grep defunct было установлено, что зомби-процессы находятся в состоянии Z и имеют метку <defunct>. У них отображается PID, PPID, но они не потребляют ресурсы процессора или оперативной памяти.

При попытке досрочно завершить зомби-процесс с помощью команды kill -9 (напрямую по PID зомби) ничего не происходит, так как процесс уже завершил свое выполнение, и ядру нечего останавливать. В рамках эксперимента был убит родительский процесс (PPID), после чего зомби-процессы мгновенно исчезли из списка. Это объясняется тем, что при завершении родителя зомби "усыновляются" процессом init (или systemd), который автоматически забирает их статус завершения.

Таким образом, зомби-процессы не опасны с точки зрения нагрузки на CPU/RAM, но их большое количество может исчерпать лимит PID в системе. Единственный способ избавиться от зомби – это дождаться завершения его родительского процесса.

Раздел 2

1. Создание скрипта cuckoo.sh:

```
#!/bin/bash

for arg in "$@"; do
    case $arg in
        --pishitesuda=*)
        PIPE_PATH="${arg#*=}"
        shift
        ;;
    esac
done

if [ -z "$PIPE_PATH" ]; then
    echo "Ошибка: не указан путь к каналу (--pishitesuda=/path/to/pipe)"
    exit 1
fi

LOG_FILE="$HOME/cuckoo.log"
mkfifo "$PIPE_PATH"

log_msg() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') $1" >> "$LOG_FILE"
}

cleanup() {
    log_msg "Shutdown!"
    rm -f "$PIPE_PATH"
    exit 0
}

trap cleanup SIGTERM SIGINT

log_msg "Startup!"

while true; do
    if read -r request < "$PIPE_PATH"; then
        if [[ "$request" =~ ^(.*)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\
?$ ]]; then
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"
            N=$(( RANDOM % 9 + 2 ))
            echo "$N" > "$PIPE_PATH"
            log_msg "$NAME[$PID] $N"
        fi
    fi
done
```

Рисунок 22 – Исходный код bash-скрипта cuckoo.sh

```
user1@eltex-practice2-pg2-v2:~$ ./cuckoo.sh --pishitesuda=/tmp/my_cuckoo_pipe
```

Рисунок 23 – Запуск скрипта cuckoo.sh с указанием пути к именованному каналу

```
user1@eltex-practice2-pg2-v2:~$ echo "Bash[12345]: how much time do I have?"
> /tmp/my_cuckoo_pipe
user1@eltex-practice2-pg2-v2:~$
user1@eltex-practice2-pg2-v2:~$ read -r answer < /tmp/my_cuckoo_pipe && echo
"Ответ: $answer"
Ответ: 3
user1@eltex-practice2-pg2-v2:~$ cat ~/cuckoo.log
2026-10-29 13:27:21 Startup!
2026-10-29 13:27:32 Bash[12345] 3
2026-10-29 13:28:13 Shutdown!
2026-10-29 13:32:13 Startup!
2026-10-29 13:32:23 Bash[12345] 2
2026-10-29 13:33:52 Bash[12345] 7
2026-10-29 13:34:10 Shutdown!
2026-10-29 13:34:13 Startup!
2026-10-29 13:34:20 Bash[12345] 3
2026-10-29 13:35:42 Shutdown!
2026-10-29 13:36:19 Startup!
2026-10-29 13:36:23 Bash[12345] 3
2026-10-29 13:36:34 Shutdown!
2026-10-29 13:36:37 Startup!
2026-10-29 13:36:42 Bash[12345] 9
2026-10-29 13:36:55 Shutdown!
2026-10-29 13:39:48 Startup!
2026-10-29 13:39:56 Bash[12345] 10
2026-10-29 13:42:14 Shutdown!
2026-10-29 13:42:16 Startup!
2026-10-29 13:42:18 Shutdown!
2026-10-29 13:42:22 Startup!
2026-10-29 13:42:57 Bash[12345] 3
2026-10-29 13:43:09 Shutdown!
```

Рисунок 24 – Проверка работы cuckoo.sh: отправка запроса через канал и просмотр лога cuckoo.log

2. Создание службы system для запуска скрипта cuckoo.sh, разрешение его работы, запуск и проверка состояния:

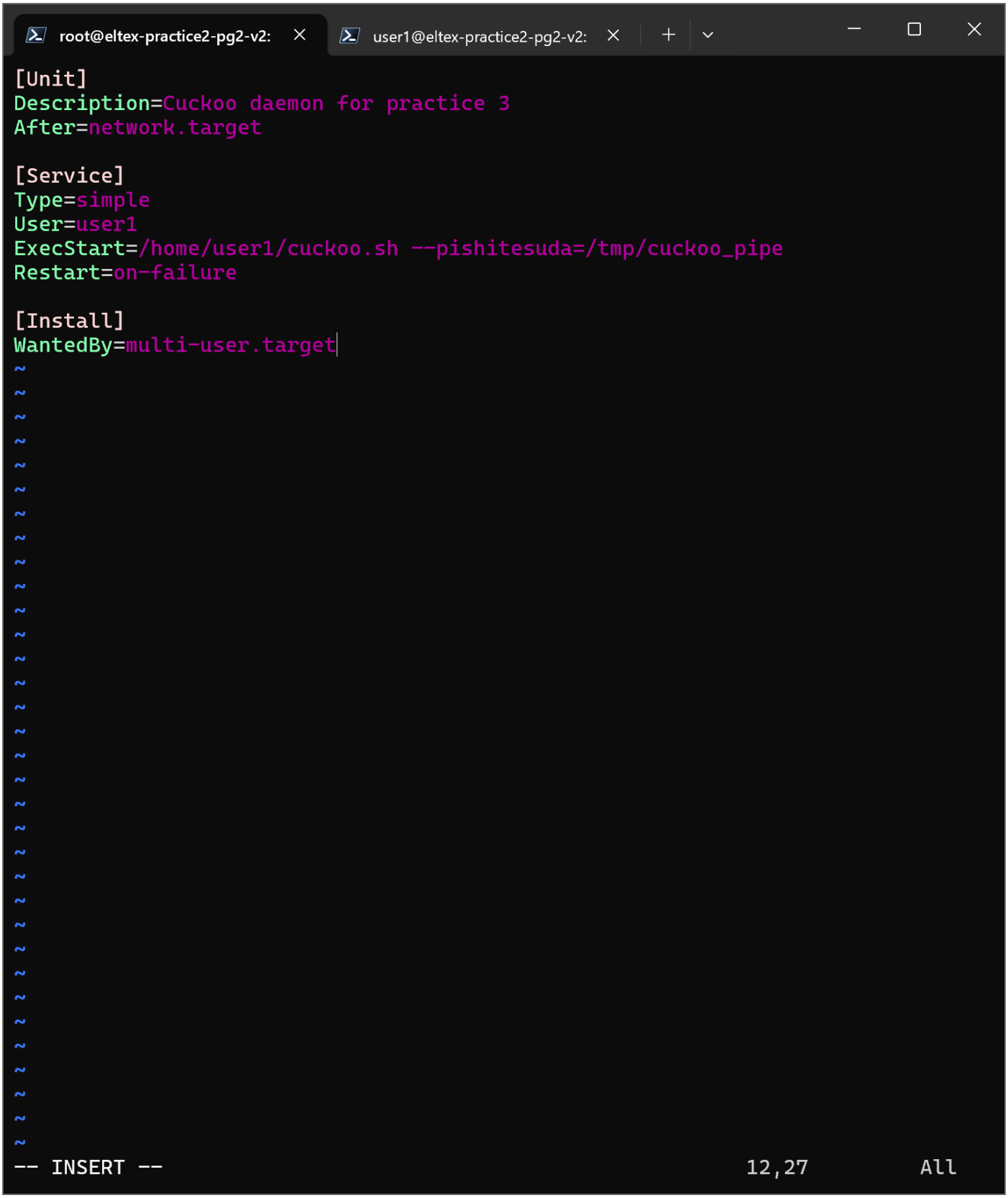


Рисунок 25 – Создание файла конфигурации службы systemd /etc/systemd/system/cuckoo.service

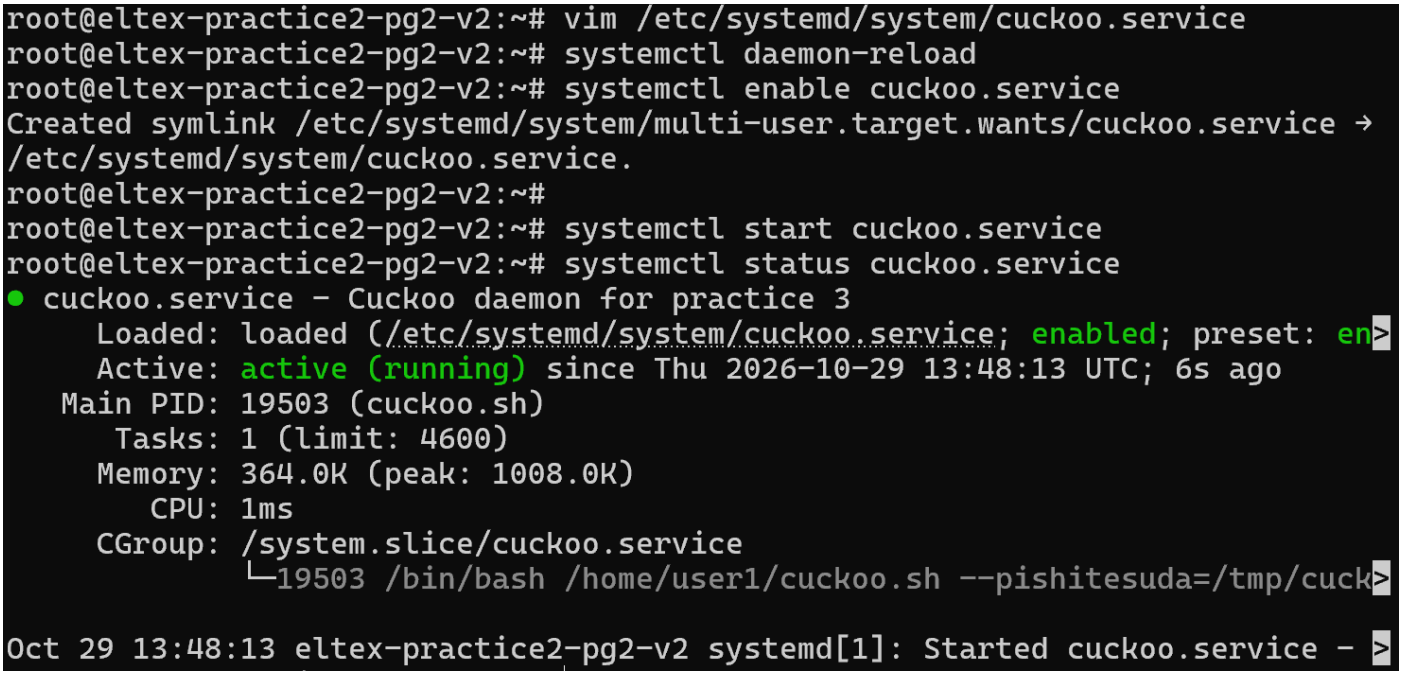


Рисунок 26 – Запуск, включение в автозагрузку и проверка статуса службы cuckoo.service

3. Создание скрипта на языке bash с именем template_task.sh:

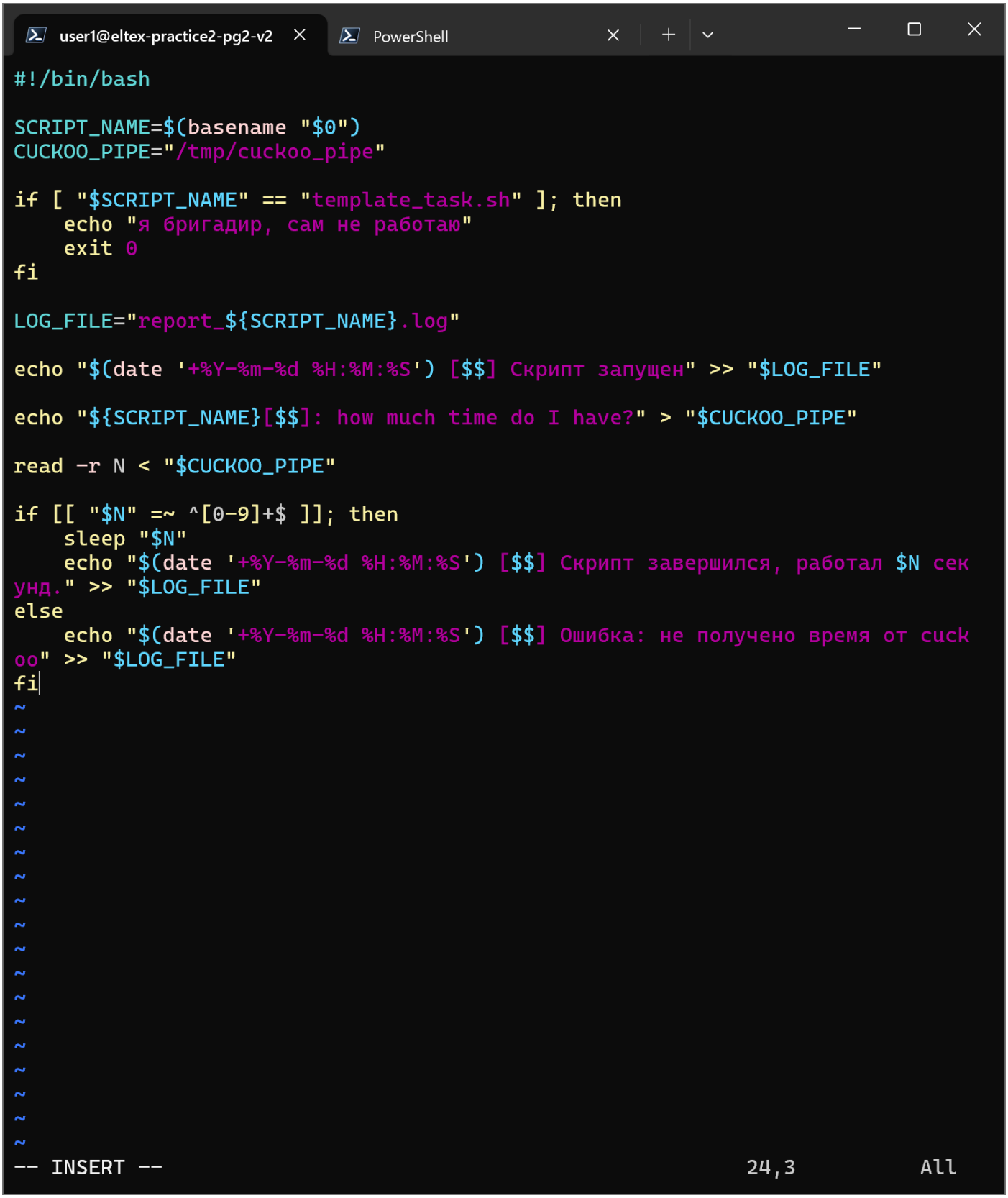


Рисунок 27 – Исходный код bash-скрипта template_task.sh

```
user1@eltex-practice2-pg2-v2:~$ vim ~/template_task.sh
user1@eltex-practice2-pg2-v2:~$ chmod +x ~/template_task.sh
user1@eltex-practice2-pg2-v2:~$ ./template_task.sh
я бригадир, сам не работаю
user1@eltex-practice2-pg2-v2:~$ ln -s ~/template_task.sh ~/task_A.sh
user1@eltex-practice2-pg2-v2:~$ ./task_A.sh
user1@eltex-practice2-pg2-v2:~$ cat ~/report_task_A.sh.log
2026-10-29 13:51:17 [19519] Скрипт запущен
2026-10-29 13:51:24 [19519] Скрипт завершился, работал 7 секунд.
```

Рисунок 28 – Создание символьной ссылки на template_task.sh, запуск и проверка файла лога

4. Создание скрипта на языке bash с именем observer.sh:

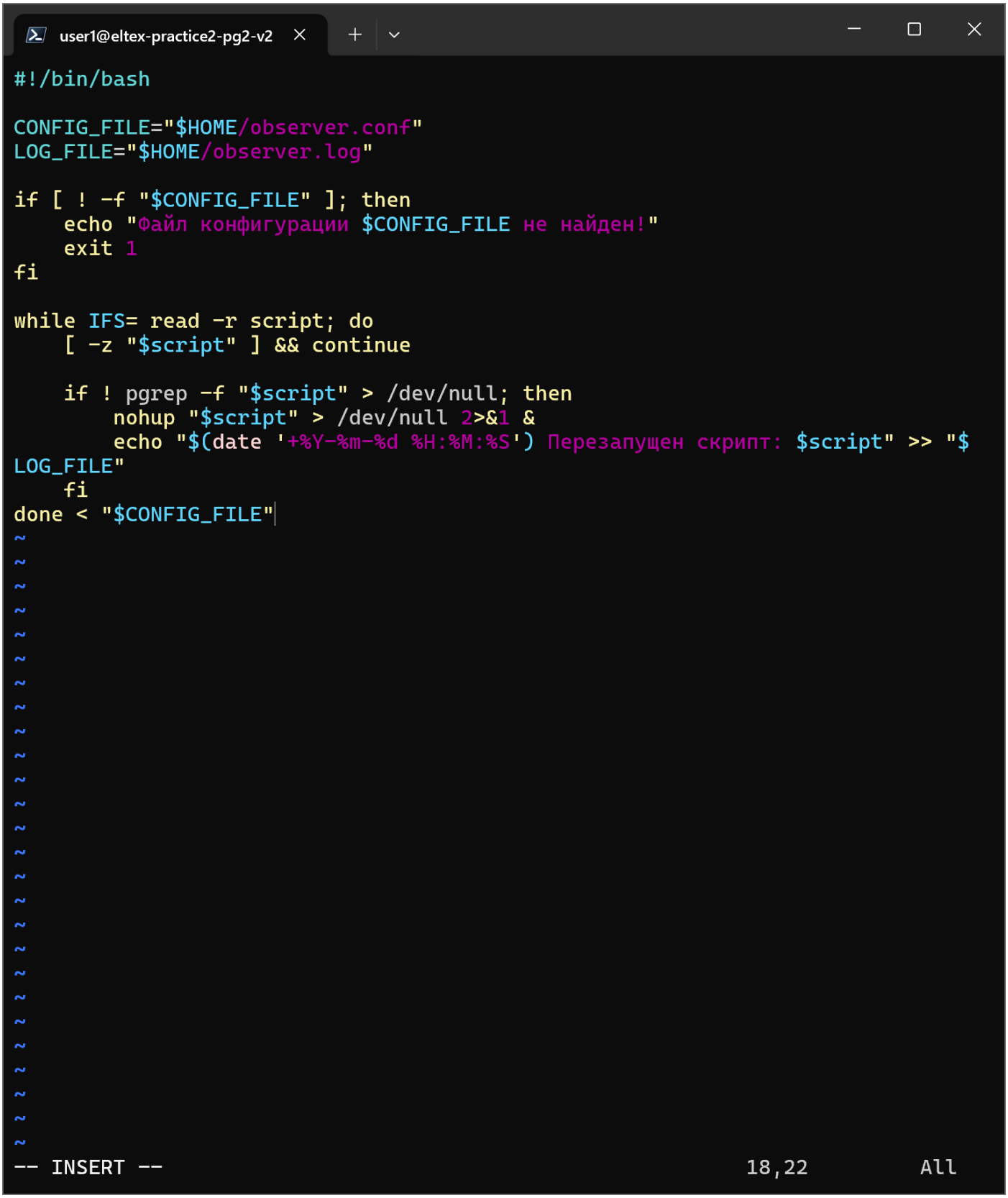


Рисунок 29 – Исходный код bash-скрипта observer.sh

```
user1@eltex-practice2-pg2-v2:~$ vim ~/observer.sh
user1@eltex-practice2-pg2-v2:~$ chmod +x ~/observer.sh
user1@eltex-practice2-pg2-v2:~$ ln -s ~/template_task.sh ~/task_B.sh
user1@eltex-practice2-pg2-v2:~$ ln -s ~/template_task.sh ~/task_C.sh
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_B.sh" > ~/observer.con
f
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_C.sh" >> ~/observer.co
nf
user1@eltex-practice2-pg2-v2:~$ ./observer.sh
user1@eltex-practice2-pg2-v2:~$ cat ~/observer.log
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_C.sh
```

Рисунок 30 – Настройка конфигурации observer.conf, ручной запуск observer.sh и проверка лога

5. Настройка запуска observer.sh посредством cron по расписанию – 1 раз в минуту:

```
user1@eltex-practice2-pg2-v2:~$ crontab -e
no crontab for user1 - using an empty one

Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny
  4. /bin/ed

Choose 1-4 [1]:
crontab: installing new crontab
user1@eltex-practice2-pg2-v2:~$ crontab -e
No modification made
user1@eltex-practice2-pg2-v2:~$ cat ~/observer.log
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 13:59:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 13:59:01 Перезапущен скрипт: /home/user1/task_C.sh
```

Рисунок 31 – Настройка запуска observer.sh посредством cron по расписанию (1 раз в минуту)

6. Создание нескольких символьных ссылок на файл template_task.sh, добавление в файл конфигурации observer.conf соответствующих записей об этих задачах, включая исходный template_task.sh:

```
user1@eltex-practice2-pg2-v2:~$ ln -s ~/template_task.sh ~/task_D.sh
user1@eltex-practice2-pg2-v2:~$ ln -s ~/template_task.sh ~/task_E.sh
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/template_task.sh" > ~/obser
ver.conf
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_B.sh" >> ~/observer.co
nf
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_C.sh" >> ~/observer.co
nf
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_D.sh" >> ~/observer.co
nf
user1@eltex-practice2-pg2-v2:~$ echo "/home/user1/task_E.sh" >> ~/observer.co
nf
user1@eltex-practice2-pg2-v2:~$ cat ~/observer.log
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 13:53:49 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 13:59:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 13:59:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:00:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:00:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:01:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:01:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:02:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:02:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:03:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:03:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:04:01 Перезапущен скрипт: /home/user1/template_task.sh
2026-10-29 14:04:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:04:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:04:01 Перезапущен скрипт: /home/user1/task_D.sh
2026-10-29 14:04:01 Перезапущен скрипт: /home/user1/task_E.sh
2026-10-29 14:05:01 Перезапущен скрипт: /home/user1/template_task.sh
2026-10-29 14:05:01 Перезапущен скрипт: /home/user1/task_B.sh
2026-10-29 14:05:01 Перезапущен скрипт: /home/user1/task_C.sh
2026-10-29 14:05:01 Перезапущен скрипт: /home/user1/task_D.sh
2026-10-29 14:05:01 Перезапущен скрипт: /home/user1/task_E.sh
```

Рисунок 32 – Добавление новых символьных ссылок в observer.conf и проверка лога перезапуска задач

```
user1@eltex-practice2-pg2-v2:~$ cat ~/report_task_B.sh.log
2026-10-29 13:53:49 [19534] Скрипт запущен
2026-10-29 13:53:49 [19534] Ошибка: не получено время от cuckoo
2026-10-29 13:59:01 [19600] Скрипт запущен
2026-10-29 13:59:11 [19600] Скрипт завершился, работал 10 секунд.
2026-10-29 14:00:01 [19622] Скрипт запущен
2026-10-29 14:00:07 [19622] Скрипт завершился, работал 6 секунд.
2026-10-29 14:01:01 [19643] Скрипт запущен
2026-10-29 14:01:08 [19643] Скрипт завершился, работал 7 секунд.
2026-10-29 14:02:01 [19662] Скрипт запущен
2026-10-29 14:02:07 [19662] Скрипт завершился, работал 6 секунд.
2026-10-29 14:03:01 [19682] Скрипт запущен
2026-10-29 14:03:10 [19682] Скрипт завершился, работал 9 секунд.
2026-10-29 14:04:01 [19708] Скрипт запущен
2026-10-29 14:04:10 [19708] Скрипт завершился, работал 9 секунд.
2026-10-29 14:05:01 [19756] Скрипт запущен
2026-10-29 14:05:06 [19756] Скрипт завершился, работал 5 секунд.
```

Рисунок 33 – Пример содержимого лог-файла задачи report_task_B.sh.log

7. Сборка статистики работы в виде набора файлов report_*.log, observer.log, cuckoo.log:

```
user1@eltex-practice2-pg2-v2:~$ tar -czvf ~/practice3_section2.tar.gz ~/repor
t_*.log ~/observer.log ~/cuckoo.log ~/cuckoo.sh ~/template_task.sh ~/observer
.sh ~/observer.conf
tar: Removing leading `/' from member names
/home/user1/report_task_A.sh.log
tar: Removing leading `/' from hard link targets
/home/user1/report_task_B.sh.log
/home/user1/report_task_C.sh.log
/home/user1/report_task_D.sh.log
/home/user1/report_task_E.sh.log
/home/user1/observer.log
/home/user1/cuckoo.log
/home/user1/cuckoo.sh
/home/user1/template_task.sh
/home/user1/observer.sh
/home/user1/observer.conf
user1@eltex-practice2-pg2-v2:~$ crontab -r
user1@eltex-practice2-pg2-v2:~$ pkill -f template_task.sh
user1@eltex-practice2-pg2-v2:~$ pkill -f observer.sh
user1@eltex-practice2-pg2-v2:~$ pkill -f task_
user1@eltex-practice2-pg2-v2:~$ su -
Password:
root@eltex-practice2-pg2-v2:~# systemctl stop cuckoo.service
root@eltex-practice2-pg2-v2:~# systemctl disable cuckoo.service
Removed "/etc/systemd/system/multi-user.target.wants/cuckoo.service".
root@eltex-practice2-pg2-v2:~# exit
logout
```

Рисунок 34 – Архивация логов (tar), удаление задачи из cron и остановка/отключение службы cuckoo.service

Приложение

1. Скрипт cuckoo.sh:

```
#!/bin/bash

for arg in "$@"; do
    case $arg in
        --pishitesuda=*)
            PIPE_PATH="${arg#*=}"
            shift
            ;;
    esac
done

if [ -z "$PIPE_PATH" ]; then
    echo "Ошибка: не указан путь к каналу (--pishitesuda=/path/to/pipe)"
    exit 1
fi

LOG_FILE="$HOME/cuckoo.log"
mkfifo "$PIPE_PATH"

log_msg() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') $1" >> "$LOG_FILE"
}

cleanup() {
    log_msg "Shutdown!"
    rm -f "$PIPE_PATH"
    exit 0
}

trap cleanup SIGTERM SIGINT

log_msg "Startup!"

while true; do
    if read -r request < "$PIPE_PATH"; then
        if [[ "$request" =~ ^(.*)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\?$ ]]; then
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"
            N=$(( RANDOM % 9 + 2 ))
            echo "$N" > "$PIPE_PATH"
            log_msg "$NAME[$PID] $N"
        fi
    fi
done
```

2. Скрипт template_task.sh

```
#!/bin/bash

SCRIPT_NAME=$(basename "$0")
CUCKOO_PIPE="/tmp/cuckoo_pipe"

if [ "$SCRIPT_NAME" == "template_task.sh" ]; then
    echo "я бригадир, сам не работаю"
    exit 0
fi

LOG_FILE="report_${SCRIPT_NAME}.log"
```

```
echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт запущен" >> "$LOG_FILE"

echo "${SCRIPT_NAME}[$$]: how much time do I have?" > "$CUCKOO_PIPE"

read -r N < "$CUCKOO_PIPE"

if [[ "$N" =~ ^[0-9]+$ ]]; then
    sleep "$N"
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт завершился, работал $N секунд." >> "$LOG_FILE"
else
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Ошибка: не получено время от cuckoo" >> "$LOG_FILE"
fi
```

3. Скрипт observer.sh

```
#!/bin/bash

CONFIG_FILE="$HOME/observer.conf"
LOG_FILE="$HOME/observer.log"

if [ ! -f "$CONFIG_FILE" ]; then
    echo "Файл конфигурации $CONFIG_FILE не найден!"
    exit 1
fi

while IFS= read -r script; do
    [ -z "$script" ] && continue

    if ! pgrep -f "$script" > /dev/null; then
        nohup "$script" > /dev/null 2>&1 &
        echo "$(date '+%Y-%m-%d %H:%M:%S') Перезапущен скрипт: $script" >> "$LOG_FILE"
    fi
done < "$CONFIG_FILE"
```